UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	M. Fadhil Anhar (NIM 2428240158)
Kelas	SI5C
Judul project / topik API	API Data Streaming (Topik 34: Streaming - Serial TV), resource /tv-series
Tautan repositori	https://github.com/anhrxf/si5c_Tugas_paw
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	16 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

> **Sebelum mengambil screenshot Runner**
> Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:
API ini adalah REST API untuk mengelola data serial TV (Topik 34: Streaming - Serial TV). Entitas utamanya adalah data streaming dengan atribut id, judul, jumlahEpisode, status, studio, dan tahunRilis, yang diakses melalui resource /tv-series dan disimpan sementara di memori server tanpa database.
API dibangun dengan Node.js dan Express.js berstruktur Route-Controller-Model. Middleware yang dipakai adalah logger, CORS, cekApiKey (header x-api-key untuk POST, PUT, dan DELETE), serta penanganan kesalahan yang selalu mengembalikan JSON.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/tv-series	Tidak	200	Mengambil seluruh data streaming
2	GET	/tv-series/:id	Tidak	200	Mengambil satu data berdasarkan ID (404 bila tidak ada)
3	POST	/tv-series	Ya	201	Menambah data baru; semua field wajib diisi (400 bila tidak lengkap)
4	PUT	/tv-series/:id	Ya	200	Memperbarui field yang dikirim; field lain tetap
5	DELETE	/tv-series/:id	Ya	204	Menghapus data berdasarkan ID
6	GET	/tv-series?status=Tamat	Tidak	200	Memfilter data berdasarkan status (variasi query dari endpoint 1)

Catatan: rute GET / (pesan server berjalan) dan GET /error-uji (simulasi kesalahan 500) adalah rute pendukung, bukan endpoint data, sehingga tidak dimasukkan ke daftar di atas.

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

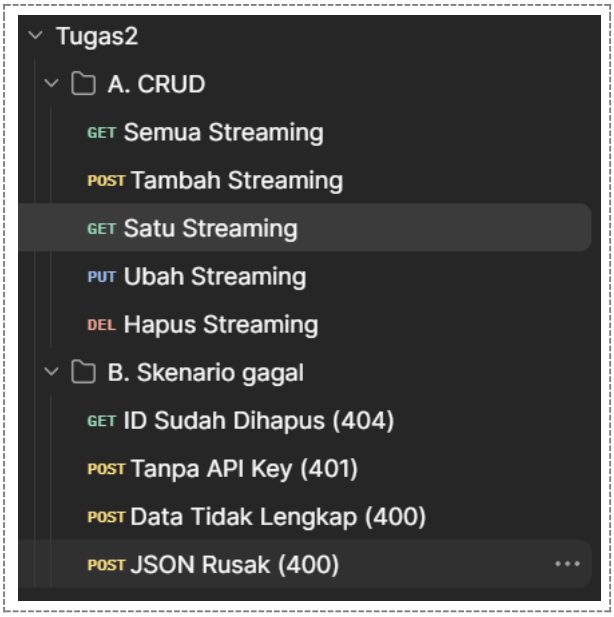
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat server
2	api-key	(sama dengan API_KEY pada berkas .env; nilai asli tidak dicantumkan)	Nilai header x-api-key untuk POST, PUT, dan DELETE
3	streamingId	(kosong, diisi script)	ID dari respons POST Tambah Streaming; dipakai request Satu, Ubah, Hapus, dan ID Sudah Dihapus

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:

Collection bernama Tugas2 dibagi menjadi dua folder. Folder "A. CRUD" berisi lima request utama berurutan: Semua Streaming, Tambah Streaming, Satu Streaming, Ubah Streaming, dan Hapus Streaming. Folder "B. Skenario gagal" berisi empat request: ID Sudah Dihapus (404), Tanpa API Key (401), Data Tidak Lengkap (400), dan JSON Rusak (400).

Urutannya disengaja. Tambah Streaming harus berjalan sebelum Satu, Ubah, dan Hapus karena request itulah yang menyimpan streamingId. Hapus Streaming ditaruh paling akhir di folder A agar request ID Sudah Dihapus di folder B dapat memakai ID yang sama dan memastikan data benar-benar sudah hilang (404). Request Semua Streaming sekaligus mencakup endpoint GET /tv-series; filter ?status= belum memiliki request tersendiri.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	Semua Streaming	GET	{{base_url}}/tv-series	A. CRUD
2	Tambah Streaming	POST	{{base_url}}/tv-series	A. CRUD
3	Satu Streaming	GET	{{base_url}}/tv-series/{{streamingId}}	A. CRUD

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
4	Ubah Streaming	PUT	{{base_url}}/tv-series/{{streamingId}}	A. CRUD
5	Hapus Streaming	DELETE	{{base_url}}/tv-series/{{streamingId}}	A. CRUD
6	ID Sudah Dihapus (404)	GET	{{base_url}}/tv-series/{{streamingId}}	B. Skenario gagal
7	Tanpa API Key (401)	POST	{{base_url}}/tv-series	B. Skenario gagal
8	Data Tidak Lengkap (400)	POST	{{base_url}}/tv-series	B. Skenario gagal
9	JSON Rusak (400)	POST	{{base_url}}/tv-series	B. Skenario gagal

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	Semua Streaming	Status code adalah 200	Response berupa array; setiap data memiliki id dan judul; data pertama berjudul Rahasia Kampus Tua. Ditambah pemeriksaan response time di bawah 500 ms
2	Tambah Streaming	Status code adalah 201	Data baru memiliki id dan judul sesuai kiriman; id disimpan ke variabel streamingId
3	Satu Streaming	Status code adalah 200	id sama dengan streamingId dan judul adalah Tunggu Aku Sukses Nanti
4	Ubah Streaming	Status code adalah 200	status berubah menjadi Tayang, judul tetap
5	Hapus Streaming	Status code adalah 204	Body respons kosong
6	ID Sudah Dihapus (404)	Status code adalah 404	Pesan error: Data tidak ditemukan
7	Tanpa API Key (401)	Status code adalah 401	Pesan error: API key tidak valid
8	Data Tidak Lengkap (400)	Status code adalah 400	Pesan error mengandung kata "wajib diisi"
9	JSON Rusak (400)	Status code adalah 400	Respons berupa JSON (bukan HTML) dengan pesan Format JSON tidak valid

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
// Test script request GET: Semua Streaming
pm.test('Status code adalah 200', function () {
  pm.response.to.have.status(200);
});

pm.test('Response berupa array', function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.be.an('array');
});

pm.test('Setiap data memiliki id dan judul', function () {
  const jsonData = pm.response.json();
  jsonData.forEach((item) => {
    pm.expect(item).to.have.property('id');
    pm.expect(item).to.have.property('judul');
  });
});

pm.test('Response time di bawah 500ms', function () {
  pm.expect(pm.response.responseTime).to.be.below(500);
});

pm.test('Data streaming pertama berjudul Rahasia Kampus Tua', function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData[0].judul).to.eql('Rahasia Kampus Tua');
});
```

Penjelasan:

Test pertama, "Status code adalah 200", memastikan endpoint GET /tv-series menjawab berhasil. Bila server mati atau rute salah, test ini gagal lebih dulu.

Test "Response berupa array" memeriksa bentuk respons berupa daftar, bukan objek tunggal. Test "Setiap data memiliki id dan judul" menelusuri tiap elemen dengan forEach dan memastikan properti id dan judul ada, sehingga struktur data konsisten bagi pemakai API.

Test "Response time di bawah 500ms" menjaga agar API tetap cepat. Test terakhir, "Data streaming pertama berjudul Rahasia Kampus Tua", memeriksa isi data awal pada model. Test ini juga menjadi penanda bahwa server dalam kondisi awal, sehingga server perlu dinyalakan ulang sebelum Runner dijalankan.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
// Test script request POST: Tambah Streaming
pm.test('Status code adalah 201', function () {
  pm.response.to.have.status(201);
});

pm.test('Data baru memiliki id dan judul sesuai kiriman', function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property('id');
  pm.expect(jsonData.judul).to.eql('Tunggu Aku Sukses Nanti');
});

if (pm.response.code === 201) {
  pm.environment.set('streamingId', pm.response.json().id);
}
```

Penjelasan:

Pada request Tambah Streaming, ID dari respons disimpan dengan pm.environment.set("streamingId", pm.response.json().id). Penyimpanan dibungkus kondisi if (pm.response.code === 201) agar ID hanya disimpan bila data benar-benar berhasil dibuat dan tidak mengambil nilai dari respons error.

Variabel streamingId dipakai oleh Satu Streaming, Ubah Streaming, dan Hapus Streaming di folder A, serta ID Sudah Dihapus (404) di folder B, lewat alamat {{base_url}}/tv-series/{{streamingId}}.

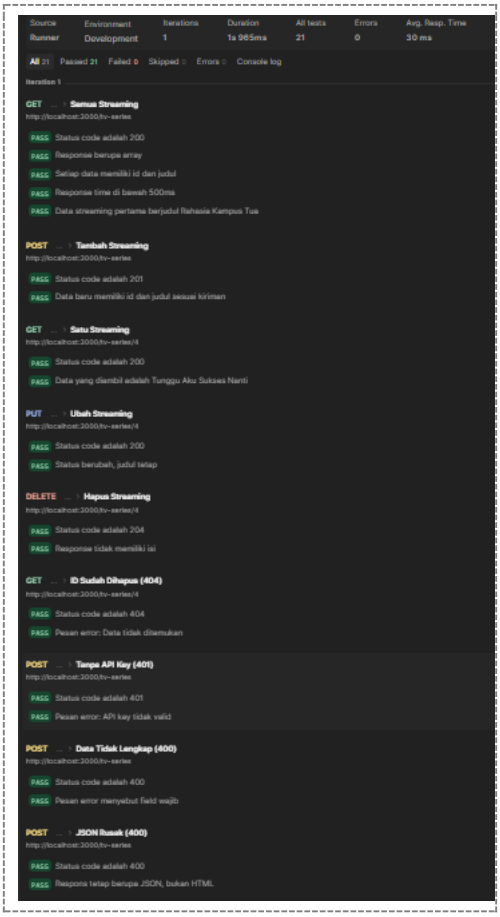
Bila urutan ditukar atau POST gagal, streamingId kosong dan alamat menjadi /tv-series/. Hal ini terjadi pada percobaan yang gagal (lihat 4.3): GET mengembalikan daftar data dengan status 200, sedangkan PUT dan DELETE mendapat 404 karena rute tidak ditemukan, sehingga banyak test ikut gagal. Request ID Sudah Dihapus juga akan menerima 200, bukan 404, bila dijalankan sebelum Hapus Streaming.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:

Collection Tugas2 dijalankan seluruhnya dengan Environment Development. Hasilnya, 9 request dieksekusi dan 21 assertion semuanya lulus (0 gagal), baik pada folder A. CRUD maupun B. Skenario gagal.

Sebelum Runner dijalankan, server dinyalakan ulang agar data kembali ke kondisi awal (tiga data awal, dan data baru mendapat id 4). Environment Development dipilih supaya {{base_url}} dan {{api-key}} terbaca.

4.2 Rekap angka hasil

Nama Collection	Tugas2
Environment yang dipakai	Development
Jumlah request dijalankan	9
Jumlah test lulus	21

Jumlah test gagal	0
Tanggal dan jam pengujian	16 Oktober 2026, pukul 19.46 WIB

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	Tambah Streaming	Status code adalah 201: expected 201 but got 401; Data baru memiliki id dan judul sesuai kiriman: expected { message: 'API key tidak valid' } to have property 'id'	Header x-api-key yang berisi {{api-key}} ditolak cekApiKey (401), sehingga data tidak dibuat dan streamingId tidak tersimpan	Environment Development dipilih dan nilai api-key disamakan dengan API_KEY pada .env, lalu server dinyalakan ulang dan Collection dijalankan ulang
2	Satu, Ubah, Hapus, dan ID Sudah Dihapus (404)	Status code adalah 200 (PUT) dan 204 (DELETE) mendapat 404; ID Sudah Dihapus: expected 404 but got 200	streamingId kosong sehingga alamat menjadi /tv-series/; GET mengembalikan daftar (200), PUT dan DELETE mendapat 404 rute tidak ditemukan	Sumber masalahnya sama dengan baris 1; setelah POST berhasil dan ID tersimpan, semua request berantai ini lulus
3	Data Tidak Lengkap (400)	Status code adalah 400: expected 400 but got 401; Pesan error menyebut field wajib: expected 'API key tidak valid' to include 'wajib diisi'	Pemeriksaan API key berjalan sebelum validasi data, sehingga request tanpa key yang benar ditolak 401 lebih dulu	Nilai api-key pada Environment diperbaiki seperti baris 1, sehingga request sampai ke validasi dan menghasilkan 400
4	Ubah Streaming (percobaan regresi)	Status berubah, judul tetap: expected undefined to deeply equal 'Tunggu Aku Sukses Nanti' (1 dari 21 assertion gagal)	Respons PUT tidak lagi memuat field judul (ukuran respons 327B dibanding 425B), yang menunjukkan PUT menimpa seluruh data dan bukan hanya field yang dikirim	Logika update pada model diperbaiki agar menggabungkan field yang dikirim dengan data lama ({ ...data lama, ...field baru, id }); setelah itu seluruh test lulus kembali

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.



Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Perintah yang dijalankan: newman run .\postman\Tugas2.postman_collection.json -e .\postman\Development.postman_environment.json

Exit code 0 berarti semua assertion lulus, seperti pada hasil akhir ini (21 assertion dieksekusi, 0 gagal, dan 9 request berjalan). Exit code 1 berarti ada minimal satu assertion yang gagal, seperti pada dua percobaan sebelumnya: 11 assertion gagal pada percobaan pertama dan 1 assertion gagal pada percobaan regresi.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/anhrxf/si5c_Tugas_paw
Lokasi berkas Collection (.json)	postman/Tugas2.postman_collection.json
Lokasi berkas Environment (.json)	postman/Development.postman_environment.json
Berkas Environment tidak memuat kunci rahasia sungguhan	Ya
Repositori dapat dibuka dosen	Ya

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:

Assertion yang paling berguna adalah "Status berubah, judul tetap" pada request Ubah Streaming. Pada percobaan regresi, hanya assertion inilah yang gagal, padahal status code tetap 200. Tanpa pemeriksaan isi JSON, kesalahan bahwa judul hilang setelah PUT tidak akan terdeteksi.

Assertion status code juga penting karena langsung menunjukkan rantai request yang putus, misalnya saat POST ditolak 401 dan request berikutnya ikut gagal.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:

Pada pengujian manual, setiap request harus dikirim satu per satu dan hasilnya dinilai dengan mata. Dengan Collection Runner, seluruh 9 request berjalan berurutan dalam sekali klik dan hasilnya langsung berupa angka lulus dan gagal, sehingga pengujian ulang setelah perbaikan jauh lebih cepat.

Bagian tersulit adalah menyambungkan request lewat variabel streamingId, karena urutan dan keberhasilan POST menentukan request lain. Saya mengatasinya dengan menyimpan ID hanya ketika status 201, memastikan Environment yang benar terpilih, dan menyalakan ulang server sebelum Runner dijalankan.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	Ya
2	Setiap endpoint API saya ada di Collection	Ya
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	Ya
4	Setiap request punya assertion status code dan struktur JSON	Ya
5	Nama setiap test berupa kalimat yang menyatakan harapan	Ya
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	Ya
7	Server dinyalakan ulang sebelum Runner dijalankan	Ya
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	Ya
9	Berkas Environment tidak memuat kunci rahasia sungguhan	Ya
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	Ya

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen